Understanding the GPU Memory Hierarchy

From Registers and Shared Memory to L2, HBM, and Data Movement

---

A technical handbook on where AI workloads keep data and how those bytes move: registers, shared memory, L1/L2 caches, HBM, locality, working sets, coalescing, tiling, and the memory behavior behind modern GPU kernels.

@techNmak

How to read this handbook

GPU memory is easiest to reason about when four questions are kept separate:

1. **What is the logical memory space?** CUDA names spaces such as register, local, shared, and global memory. Those names describe visibility, lifetime, and programming semantics, not a simple physical ladder.[1]
2. **Where can the data be served from?** A global load may be satisfied by the SM-side L1/TEX path, by the GPU-wide L2 cache, or by device memory. A local-memory access is thread-private in scope but still uses device-memory backing.[1][3]
3. **How many bytes actually move?** Tensor size is capacity. Traffic is the amount transferred across a particular interface. The two can differ by orders of magnitude when data is reused or repeatedly fetched.
4. **What is limiting progress?** Capacity, latency, bandwidth, request throughput, instruction issue, synchronization, and compute throughput are different constraints. A kernel can fit comfortably in HBM and still spend most of its time waiting on memory movement.

Core distinction. A memory hierarchy is not merely a list from "fast" to "slow." Performance depends on which bytes move, how often they move, which level supplies them, and whether enough independent work exists to overlap the wait.

Scope. This handbook establishes the memory mental model needed by the rest of Understanding AI Systems. Arithmetic intensity, Roofline analysis, detailed coalescing rules, shared-memory bank conflicts, register/occupancy tuning, kernel fusion, and profiling each receive dedicated handbooks later. Here they appear only far enough to make memory behavior coherent.

Vendor convention. NVIDIA CUDA terminology is used for concrete examples because it is common in AI systems. AMD CDNA terminology is used as a cross-check so that general principles are not presented as CUDA laws.[12][16]

Units. This handbook uses KiB, MiB, and GiB when a calculation explicitly uses binary powers of two. Product specifications are reproduced using the units printed by the vendor, such as GB and TB/s, rather than silently converting the labels.[2]

Contents

1	Why a GPU needs a memory hierarchy	5
2	The hierarchy at a glance	6
3	Capacity, traffic, bandwidth, and latency	7
4	Registers: the closest working state	8
5	Register files are finite resources	9
6	Local memory and register spills	10
7	Shared memory: explicit on-chip storage	11
8	Shared memory is not just another cache	12
9	L1: a small, architecture-dependent first cache	13
10	L2: device-wide reuse before HBM	14
11	HBM and device memory	15
12	HBM is not a synonym for global memory	16
13	A load's journey through the hierarchy	17
14	Temporal locality	18
15	Spatial locality	19
16	Working sets and reuse distance	20
17	Cache hits, misses, and downstream traffic	21
18	Coalescing: organize a warp's memory requests	22
19	Worked example: contiguous and strided loads	23
20	Tiling turns capacity into reuse	24
21	GEMM is a memory-reuse problem too	25
22	Bytes moved versus arithmetic performed	26
23	Why elementwise kernels and GEMM behave differently	27
24	Attention is also a data-movement problem	28

Contents continued

25	FlashAttention as a hierarchy-aware example	29
26	Prefill and decode stress memory differently	30
27	Weights, activations, and KV state have different lifetimes	31
28	Training and inference move different bytes	32
29	Capacity and bandwidth are independent hardware properties	33
30	Theoretical bandwidth versus achieved bandwidth	34
31	Cache-friendly does not automatically mean fast	35
32	High bandwidth does not mean low latency	36
33	NVIDIA and AMD: same problem, different vocabulary	37
34	Architecture generations change the details	38
35	Common misconceptions	39
36	A practical memory mental model	40
	References and further reading	41
	Reference map: what to read for which question	42

Chapters 1–32 focus on durable concepts. Chapter 33 translates terminology across vendors. Chapter 34 is deliberately generation-specific and should be rechecked when hardware documentation changes.

1 Why a GPU needs a memory hierarchy

A GPU can execute an enormous number of arithmetic operations, but arithmetic units can only work on data that has reached them. The storage system therefore has to solve conflicting goals at once: keep enough total bytes for large models, deliver those bytes at high aggregate rates, and make frequently reused values available close to the execution units.

No single memory technology does all three well. Very large memory pools are physically farther from the arithmetic pipelines and are expensive to provision at the same access characteristics as tiny on-chip storage. Very small structures near the pipelines can be extremely useful, but they cannot hold an entire model or even a large activation tensor.

Modern GPUs therefore use layers of storage. Thread-private values commonly live in registers. Threads in a block can explicitly cooperate through shared memory. Hardware-managed caches capture some reuse without explicit staging. Device memory, often HBM on datacenter AI accelerators, provides much larger capacity and very high aggregate bandwidth.[1][5][6]

The important word is *reuse*. If a value is fetched from device memory and used once, the hierarchy has little opportunity to amortize that movement. If the same value can be used dozens or hundreds of times while it remains on chip, a relatively expensive transfer can feed a large amount of computation.

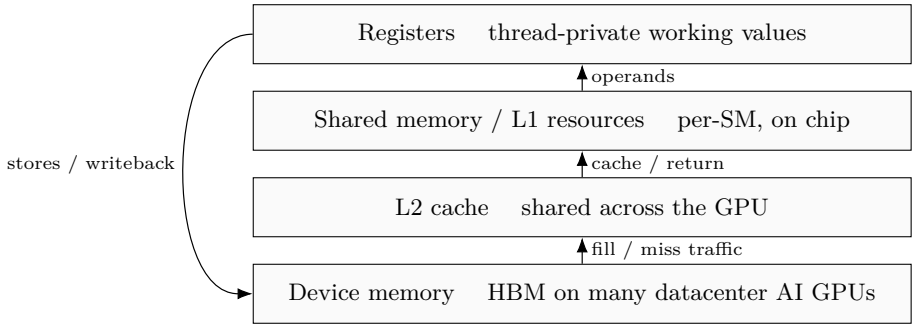
This is why two kernels with the same mathematical operation count can run at very different speeds. One mapping may continually fetch data from device memory. Another may reorganize the work so that each fetched tile participates in many operations before it is replaced.

Mental model. GPU memory performance is largely a question of placing useful reuse at the right level of the hierarchy while avoiding unnecessary traffic between levels.

The rest of this handbook builds that model from the storage closest to a thread outward to device memory.

2 The hierarchy at a glance

The diagram below is intentionally schematic. It shows a common path for AI data, not a promise that every instruction on every GPU follows exactly these boxes.



Three cautions keep this picture honest.

First, **programming spaces and physical structures are not identical**. CUDA global memory is an address space visible across the grid. A global load can hit an on-chip cache. CUDA local memory is private to one thread, yet its backing storage is in device memory.[1][3]

Second, **the hierarchy is architecture-dependent**. On recent NVIDIA datacenter GPUs, the L1/texture cache and shared memory draw from a unified on-chip resource whose partition can be adjusted. AMD CDNA exposes a different organization with per-CU L1 caches and Local Data Share (LDS).[5][6][12]

Third, **closer does not mean automatically better**. Moving data into shared memory costs instructions and often synchronization. Keeping more values in registers can increase register pressure. A cache may contain data that is never reused. Each level helps only when the access pattern makes its cost worthwhile.

> Think of the hierarchy as a set of places where data can be retained and reused, not as a universal ranking with one fixed latency number per level.

3 Capacity, traffic, bandwidth, and latency

These four quantities are often collapsed into the vague word "memory." They answer different questions.

Capacity is how many bytes a storage level can hold. A tensor with shape $[4, 4096, 8192]$ in BF16 contains

$$4 \times 4096 \times 8192 \times 2 = 268{,}435{,}456 \text{ bytes} = 256 \text{ MiB}.$$

That is a storage requirement, not a statement about how much data will move while a kernel runs.

Traffic is the number of bytes transferred across a particular interface. A 256 MiB tensor read ten times can create far more than 256 MiB of downstream traffic if the reads are not served by a cache or other reuse mechanism.

Bandwidth is a rate:

$$\text{bandwidth} = \frac{\text{bytes transferred}}{\text{time}}.$$

Peak bandwidth is a hardware ceiling under specified conditions. Achieved bandwidth depends on the access pattern, request concurrency, cache behavior, contention, and the kernel itself.[2][3]

Latency is the delay between initiating an access and having its result available to the dependent work. A system can have high bandwidth while an individual access still has substantial latency. GPUs tolerate this partly by keeping many independent operations in flight and scheduling other work while some requests wait.

Quantity	Question it answers
Capacity	How much data can reside here?
Traffic	How many bytes crossed this interface?
Bandwidth	How quickly can bytes cross over time?
Latency	How long until a particular dependency can be satisfied?

A workload can fit in HBM and still be bandwidth-limited. It can achieve high bandwidth and still suffer from latency when it exposes too little independent work. Capacity, traffic, bandwidth, and latency should never be used as synonyms.

4 Registers: the closest working state

Registers hold the values a thread needs directly for instruction execution: loop counters, pointers, fragments of matrix tiles, intermediate arithmetic results, and accumulators. CUDA automatic scalar variables generally reside in registers unless the compiler chooses another placement.[1]

Registers are not an addressable cache that threads browse collectively. They belong to the execution state of individual threads. One thread cannot simply load another thread's register value through an ordinary memory address. Communication between threads therefore uses other mechanisms such as shared memory, warp-level operations, or architecture-specific collective instructions.

This makes registers ideal for very fine-grained reuse. Consider a thread accumulating a partial dot product:

$$a \leftarrow a + x_i y_i.$$

Keeping a in a register means each multiply-accumulate can update the same nearby state without writing the partial sum to device memory after every iteration.

Matrix kernels push this much further. CUTLASS describes warp- and thread-level GEMM structures in which operands are loaded from shared memory into register-backed fragments and accumulators remain in registers across many matrix instructions.[14]

The benefit is not that "registers are free." They are a finite hardware resource and they participate in scheduling constraints. But when a value has high reuse inside one thread, keeping it in register state avoids repeated traffic through larger memory structures.

Safe wording. Registers are the normal home for thread-private working values close to execution. Avoid universal claims such as "a register access is always one cycle." Exact timing depends on the instruction path, dependencies, banking, scheduling, and architecture.

This handbook treats registers as a memory-hierarchy level because they determine whether temporary state remains close to compute or spills into the addressable memory system. The occupancy consequences receive a dedicated treatment in AS09.

5 Register files are finite resources

A register file can be large in aggregate and still become a constraint because thousands of resident threads share it. NVIDIA's current compute-capability tables list 64K 32-bit registers per SM for several recent architectures, while Hopper H100 allows at most 255 registers per thread.[4][5]

Suppose a kernel uses 128 registers per thread and launches 256-thread blocks. Ignoring allocation granularity and other architectural limits for the moment, one block would request

$$128 \times 256 = 32{,}768$$

32-bit registers. A 64K-register SM cannot host an arbitrary number of such blocks simultaneously. Register demand therefore affects how much work can be resident even when shared-memory use is tiny.

This creates a real trade-off. More registers per thread can keep useful temporaries and accumulators close to the arithmetic pipelines. But a larger register footprint can reduce the number of resident warps, which can reduce the pool of independent work available to hide stalls.

The opposite strategy can also backfire. Forcing a compiler to use fewer registers may increase occupancy, yet create extra local-memory traffic because values no longer fit in register state. In that case a kernel may gain more resident warps but do more memory work per warp.

> Register pressure is not inherently bad. It is a resource trade-off. The relevant question is whether the register footprint enables useful reuse and instruction-level parallelism without creating enough spilling or residency loss to hurt the kernel.

This is why "use fewer registers" and "maximize occupancy" are not general optimization rules. AS09 will treat that interaction directly. Here the memory lesson is simpler: a finite register budget decides how much thread-private state can remain at the closest level of the hierarchy.

6 Local memory and register spills

CUDA's term *local memory* is easy to misread. "Local" describes scope: each thread has its own local address space. It does not mean the storage is physically next to that thread's execution units. Current CUDA documentation places local memory in device memory.[1]

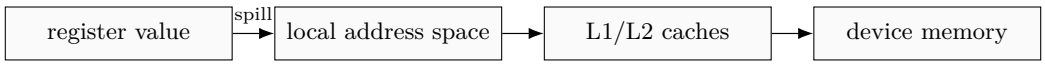
The compiler may use local memory for arrays with dynamic indexing, large thread-private objects, or values that cannot remain in registers. The last case is usually called **register spilling**.[1][3]

Conceptually:

$$\text{thread temporary} \xrightarrow{\text{does not remain in registers}} \text{local address space.}$$

A local-memory access can be served by cache when the data is present there. On a miss, however, the request travels farther through the hierarchy toward device memory. That is why a spill is not equivalent to "one HBM access every time," but it can still add loads, stores, cache pressure, and long-latency dependencies that did not exist when the value stayed in a register.

Thread-private scope does not imply a scattered physical request. CUDA lays out local storage across lanes so that a warp whose threads access the same per-thread offset can still generate a coalesced access. For example, every lane reading the same slot of its own spilled array can be serviced efficiently even though each lane owns a distinct logical object.[1]



Do not draw "local memory" next to registers in a physical hierarchy merely because it is thread-local. Its logical scope is local. Its backing storage is in the device-memory system.

7 Shared memory: explicit on-chip storage

Shared memory gives threads in a block an explicitly managed place to exchange and reuse data. CUDA exposes it with block scope and places the storage on the SM. On recent NVIDIA GPUs it is carved from the same unified on-chip resource used by L1/texture caching.[1][5][6]

That explicit control is the defining property. A hardware cache decides automatically which lines to retain. Shared memory is filled, indexed, synchronized, and overwritten according to the kernel's program logic.

A common pattern is:

1. cooperatively load a tile from device memory,
2. synchronize so the tile is ready,
3. reuse the tile across many operations,
4. replace it with the next tile when the work is finished.

The payoff comes from reuse. If 256 threads each need values that many neighboring threads also need, fetching one shared tile can replace a large number of redundant downstream loads.

Shared memory also has costs. It consumes finite per-SM capacity, can reduce block residency, requires correct synchronization when threads communicate, and has banked access behavior that can serialize some patterns. Those bank-conflict details belong to AS08.

Hopper H100 and Blackwell B200 both support configurable shared-memory carveouts within a 256 KB unified L1/texture/shared resource, with up to 228 KB of shared-memory capacity per SM and up to 227 KB addressable by one opted-in block on those specific devices.[5][6]

General principle, specific implementation. "Shared memory is programmer-managed on-chip storage" is durable. Exact capacity, carveout choices, bank behavior, and cluster-level features depend on GPU generation.

8 Shared memory is not just another cache

Shared memory and cache can both keep data close to compute, but they solve the problem differently.

	Shared memory	Hardware cache
Placement	Kernel chooses what to stage	Hardware reacts to accesses
Addressing	Explicit shared-memory address	Ordinary address space
Lifetime	Managed by kernel/block logic	Managed by replacement policy
Communication	Can exchange data among cooperating threads	Primarily serves memory accesses
Main risk	Synchronization, bank conflicts, capacity pressure	Misses, eviction, limited locality

Calling shared memory "a cache" is sometimes useful as an analogy because programmers often use it to retain data that would otherwise be fetched repeatedly. But the analogy becomes misleading if it hides the software responsibility.

For example, a matrix kernel may know exactly which A and B tiles will be used by a thread block. Staging those tiles in shared memory creates deterministic reuse. A general hardware cache does not receive that high-level intent. It sees a stream of memory requests and applies its own policies.

On the other hand, explicit staging is not always worth doing. If data has little reuse, if accesses already hit well in cache, or if the staging adds too much synchronization and copying, a shared-memory version can be slower than a simpler cached path.

> The right distinction is control. Shared memory exposes a software-controlled reuse mechanism. Cache exposes a hardware-controlled reuse mechanism. They may share physical resources on some architectures, but their programming semantics remain different.

This distinction becomes important in FlashAttention, GEMM, transposes, reductions, and many fused kernels. The kernel writer is deciding which intermediates deserve predictable on-chip residence rather than hoping a cache replacement policy keeps them around.

9 L1: a small, architecture-dependent first cache

L1 is the first hardware-managed cache encountered by many ordinary memory paths near an SM. Its exact organization changes across architectures, and different memory spaces can interact with it differently. Nsight Compute therefore presents L1/TEX as an architectural unit rather than pretending there is one universal textbook cache design.[3]

On recent NVIDIA datacenter architectures, the L1/texture cache and shared-memory capacity are part of a unified on-chip resource. Hopper H100 and Blackwell B200 expose the same 256 KB maximum combined capacity, while the amount reserved for shared memory can be changed per kernel within supported carveout choices.[5][6]

A useful L1 hit can reduce traffic to L2 and device memory. That matters because downstream interfaces are shared by more requesters and are farther from the issuing SM. Nsight's memory analysis reports requested sectors, misses to L2, hit rates, and link utilization precisely because a high-level "load count" does not reveal how much traffic escaped the first cache.[3]

But L1 hit rate is not a universal performance score. A kernel can have a high hit rate and still be limited by instruction issue, serialization, shared-memory conflicts, or some other resource. Conversely, a streaming kernel may intentionally have little temporal reuse and still perform well if its accesses are coalesced and device bandwidth is used efficiently.

Architecture warning. Do not memorize one L1 size or one caching rule as "the GPU L1." The durable idea is that a small on-chip hardware-managed structure can satisfy some requests before they consume L2 or HBM traffic. Exact capacity and behavior must be checked for the target architecture.

10 L2: device-wide reuse before HBM

L2 sits deeper in the hierarchy and is shared across the GPU rather than belonging to one SM. Nsight describes it as the cache between on-chip memory clients and device memory, and reports L2 requests, sectors, hit rate, throughput, and misses that continue to device, system, or peer memory.[3]

This position makes L2 useful for reuse that crosses block or SM boundaries. A tile evicted from one SM's L1 may still be available in L2 when another block needs the same region. Repeated model-weight reads, shared lookup tables, and overlapping tensor tiles can therefore benefit even when software does not explicitly coordinate the reuse.

L2 is still finite. If a working set is much larger than the cache, or if many unrelated streams compete for it, useful lines may be evicted before they are reused. A 64 MiB hot region, for example, cannot reside in its entirety inside H100's nominal 50 MB L2. That does not imply zero cache hits. It simply means the entire region cannot be retained simultaneously.[5]

Starting with Ampere-class CUDA devices, software can also provide persistence hints for selected global-memory regions in L2. These mechanisms influence replacement priority rather than converting L2 into explicitly addressed scratchpad memory.[1]

> L2 can reduce HBM traffic without changing the logical tensor size. This is why "the tensor is 4 GB" tells you almost nothing about how many bytes will actually cross the HBM interface during a kernel.

The next level is device memory itself, where large capacity and enormous aggregate bandwidth coexist with much higher movement cost than on-chip reuse.

11 HBM and device memory

High Bandwidth Memory, or HBM, is a physical memory technology used by many datacenter accelerators. It is designed to provide very wide interfaces and high aggregate bandwidth close to the GPU package. H100 SXM, for example, is specified with 80 GB of HBM3 and 3.35 TB/s of memory bandwidth. H200 SXM raises this to 141 GB of HBM3e at 4.8 TB/s.[7][8]

Those numbers are enormous compared with ordinary CPU memory channels, but HBM is still the far end of the on-device hierarchy from an SM's perspective. A kernel that repeatedly moves the same values between HBM and compute can become limited by that traffic even when the arithmetic units are capable of much more work.

HBM capacity and HBM bandwidth are independent specifications. More capacity lets a larger model or working set remain on the device. More bandwidth lets the memory system sustain a higher rate of transfers. Increasing one does not mathematically imply the other.

HBM is also not universal. Some GPUs use GDDR rather than HBM. The programming concept to keep stable is **device memory**: the large off-chip memory backing global allocations and other address spaces. HBM is one implementation of that role in many current AI accelerators.

> A model fitting in HBM answers a capacity question. It does not answer how many times the weights, activations, or KV state will be read, whether the accesses will hit cache, or whether the required traffic can be sustained within the latency target.

This distinction is especially important in inference, where a model may fit comfortably in memory yet still be limited by repeatedly streaming weights or KV state during decode.

12 HBM is not a synonym for global memory

CUDA's *global memory* is a programming-model address space visible across threads. HBM is a physical memory technology. Conflating them produces several wrong conclusions.

A load from global memory does not necessarily reach HBM. If the requested data is already present in a suitable cache, the request can be satisfied earlier in the hierarchy. Conversely, data from the thread-private local address space can ultimately be backed by the same device memory used for global allocations.[1][3]

The distinction can be written as:

$$\text{address-space semantics} \neq \text{where this particular access is physically served.}$$

This matters when interpreting profilers. A kernel can execute many global-load instructions while generating much less HBM traffic because L1 or L2 supplies repeated requests. The reverse can also happen: poorly coalesced or low-reuse accesses may produce far more downstream traffic than the useful bytes implied by the source code.

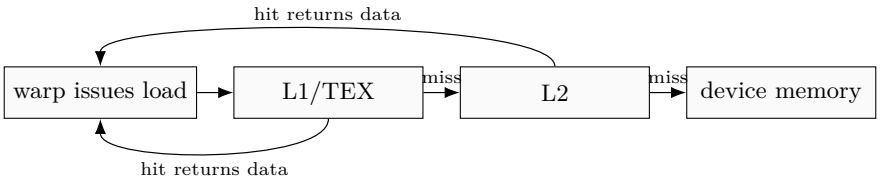
It also matters when discussing "memory usage." A model's allocated global-memory footprint is a capacity measure. Cache occupancy is a transient hardware state. Register and shared-memory use are per-kernel resource requirements. They should not be added together casually as if they were one flat pool.

Practical language. Say "global-memory load" when discussing the CUDA address space. Say "L1 hit," "L2 hit," or "HBM/device-memory traffic" when discussing where bytes were physically served or transferred.

The rest of this handbook uses that distinction consistently. It lets us talk about a single tensor while separately reasoning about its logical location, cache behavior, and actual movement through hardware.

13 A load's journey through the hierarchy

For an ordinary cacheable global load on a recent NVIDIA GPU, a useful teaching path is:



This is a mental model, not a universal routing diagram. Cache operators, texture/constant paths, asynchronous copy mechanisms, peer memory, and architecture-specific features can change the route.[1][3]

The important point is what a miss means. If L1 cannot supply the requested sector, the request consumes capacity and bandwidth farther downstream. If L2 also misses, device memory must provide the data. A cache hit therefore saves downstream traffic and usually avoids some of the delay associated with going farther out.

Multiple warps and multiple requests can be in flight at the same time. The scheduler can issue unrelated work while a dependency waits. This is how a GPU can tolerate substantial access latency without reducing the physical latency of the request itself.

A profiler therefore needs more than "load count." Nsight exposes requests, sectors, hit rates, bytes, and utilization across L1, L2, and DRAM interfaces. Those counters help answer where traffic expanded, where reuse was captured, and which link approached its throughput ceiling.[3]

> When a kernel is slow, ask two separate questions: *How far did the important requests travel?* and *Was there enough independent work to overlap the wait?*

14 Temporal locality

Temporal locality means data is reused after it was accessed recently enough that keeping it nearby may pay off.

Consider a weight tile used by several thread blocks over a short interval. The first request may fetch it from device memory. If the tile remains in L2, later blocks can reuse it without another HBM read. The tensor itself has not changed. Only the location of the reusable copy has.

The same principle appears at smaller scales. A value loaded into a register may participate in many arithmetic instructions. A shared-memory tile may feed many threads. A cache line may satisfy multiple accesses before eviction.

Temporal locality is useful only when the reuse arrives before the storage level needs the space for something else. This is why cache capacity and the surrounding access stream matter. Reusing one value after touching a huge amount of unrelated data may be too late for a small cache to retain it.

A simple way to reason about temporal reuse is to ask:

- How soon will this value be needed again?
- How much other data competes for the same storage before then?
- Is the reuse visible to hardware cache policy, or should software stage the value explicitly?

> Temporal locality is not "the same address appears twice." It is reuse that occurs on a timescale and working set where some level of the hierarchy can realistically retain the data.

AI workloads contain temporal locality at many scales: repeated weights across tokens and batch elements, reused matrix tiles inside a kernel, normalization parameters shared across positions, and KV state read across decoding steps. Which level captures that locality depends on the size and timing of the reuse.

15 Spatial locality

Spatial locality means nearby addresses are accessed together closely enough that fetching data in blocks is useful.

Caches and memory interfaces do not generally move one arbitrary scalar from HBM for every source-level load. They transfer aligned sectors or larger blocks. Current Nsight documentation, for example, reports 32-byte sectors in L1 and L2, while L2 requests may cover up to four sectors from a 128-byte cache line.[3]

If a warp's threads need neighboring 4-byte values, those useful bytes can share a small number of sectors. If the same 32 threads touch addresses spread far apart, the hardware may need many sectors even though the source code still contains one load instruction per thread.

Spatial locality therefore affects the ratio

$$\eta_{\text{traffic}} = \frac{\text{useful bytes requested by the program}}{\text{bytes transferred by the memory system}}.$$

This is not a universal hardware metric, but it is a useful accounting idea. A low ratio means movement is being spent on bytes the warp does not use.

Spatial locality is different from temporal locality. A kernel may stream perfectly contiguous data once, giving excellent spatial behavior but almost no temporal reuse. Another kernel may repeatedly revisit a small set of scattered locations, giving temporal reuse without a simple contiguous pattern.

> The hierarchy rewards both kinds of locality for different reasons: spatial locality makes each transfer more useful, while temporal locality makes the same transferred data useful again before it is evicted.

Coalescing is the GPU-specific mechanism that turns a warp's set of global-memory requests into as few memory transactions as the access pattern allows.

16 Working sets and reuse distance

A **working set** is the subset of data that needs to remain conveniently available over some phase of execution. The relevant size depends on the question: a thread's live scalars, a block's shared tile, an SM's active data, or a kernel's repeatedly reused global region are all working sets at different scales.

Cache behavior becomes easier to reason about when you compare the active working set with the storage available to retain it.

Suppose a kernel repeatedly cycles through 8 MiB of data and the relevant shared cache has ample effective space for that region. Reuse may be captured well. If the same kernel cycles through hundreds of megabytes while many other clients compete for the cache, a larger fraction of requests will need to go farther out.

Reuse distance is a useful intuition for the same idea. Between two accesses to the same value, how much distinct competing data is touched? The larger that intervening footprint, the less likely a small cache is to retain the value. Exact cache behavior also depends on mapping, associativity, replacement policy, and concurrent traffic, so reuse distance is not a complete predictor.

This explains why "fits in L2" should be used carefully. A 40 MB tensor and a 50 MB L2 do not guarantee near-perfect reuse. Other data, cache-line mapping, and simultaneous clients matter. Likewise, a working set larger than L2 can still achieve many hits if its hot subset is small or if access patterns repeatedly reuse some regions.

> A capacity comparison gives a necessary clue, not a cache-performance proof. Working-set size, reuse order, mapping, and competition together determine how much reuse is actually captured.

This is also why performance can change sharply with tensor shape even when the mathematical operation stays the same: the shape can move the active working set across a cache-capacity threshold.

17 Cache hits, misses, and downstream traffic

A cache hit is valuable because it can satisfy a request without sending the same request to a farther level. The primary memory consequence is therefore **traffic filtering**.

If 1 TB of requests arrive at L2 and a large fraction are served there, HBM may see far less than 1 TB of read traffic. If most requests miss, HBM sees much more of the demand. Nsight's memory tables expose hit rates and sector misses specifically to make this distinction visible.[3]

Hit rate by itself is still incomplete. Consider two kernels:

- Kernel A makes 10 million requests with a 95% hit rate.
- Kernel B makes 1 million requests with an 80% hit rate.

Kernel A has the better percentage but can still send more misses downstream in absolute terms. The amount of useful work performed per request also matters.

Latency behavior can differ too. A hit at one cache level may be faster than a miss that travels farther, but the exact penalty depends on architecture and request type. Some specialized cache paths, such as texture fetches, have semantics that should not be generalized to ordinary global loads.[1][3]

> When evaluating a cache, look at **bytes and requests saved downstream**, not only the hit-rate percentage. A high hit rate is useful when it removes traffic or latency from the path that was actually limiting the kernel.

This framing becomes especially important in AI kernels whose data volumes are enormous. A modest improvement in reuse can remove gigabytes of HBM traffic, while a spectacular hit-rate percentage on a tiny side structure may have almost no effect on wall-clock time.

18 Coalescing: organize a warp's memory requests

When threads in a warp issue a global-memory instruction, the GPU combines their addresses into the memory transactions needed to satisfy the warp. Good coalescing means those transactions contain a high fraction of bytes the threads actually use.[1][2]

Current CUDA examples model global-memory service in 32-byte transactions. For a concrete 64-bit case, a full warp loading 32 aligned consecutive 8-byte values needs 256 useful bytes. Those addresses occupy eight 32-byte segments.[1][3]

Lane numbering is not the rule. If the lanes access the same 32 values in a different order, the set of required segments can remain unchanged. Coalescing is therefore about the address footprint created by the warp, not about forcing lane i to read element i.[1][2]

Poor coalescing wastes movement. If those 32 useful 8-byte values are spread so that each falls in a different 32-byte segment, the request footprint grows to 32 segments even though the useful payload is still 256 bytes.

Alignment matters because a contiguous request that straddles an additional segment can require an extra transaction. Cache behavior can soften the cost of some misalignment, but it does not make scattered access patterns free.[2]

Coalescing is a traffic-efficiency concept. It reduces the number of memory segments needed for a warp's request. It should not be simplified to "threads must always access consecutive addresses."

AS07 will treat transactions, alignment, stride, and architecture-specific coalescing rules in detail. Here we need only enough to see why tensor layout can change memory traffic without changing tensor size.

19 Worked example: contiguous and strided loads

Assume one warp has 32 active threads, each loads one aligned 8-byte value, and the access footprint is counted in 32-byte segments.[1][3]

Case A: stride 1. Thread i reads element i.

$$32 \times 8 = 256 \text{ useful bytes.}$$

The addresses occupy eight 32-byte segments, so 256 segment bytes cover 256 useful bytes.

Case B: stride 2. Thread i reads element $2i$. The requested values occupy 16 segments. The warp still needs 256 useful bytes, while the covered segment footprint is 512 bytes.

Case C: stride 4. Thread i reads element $4i$. Each requested 8-byte value begins in a different 32-byte segment, so 32 segments cover 1024 bytes for 256 useful bytes.

Pattern	Segments	Segment bytes	Useful fraction
stride 1	8	256 B	100%
stride 2	16	512 B	50%
stride 4	32	1024 B	25%

These counts describe the warp's segment demand in this simplified aligned example. Actual HBM traffic can be lower if requests hit in cache, and architecture-specific paths can have additional details. The lesson is about request geometry, not a promise that every segment becomes a DRAM transaction.

The source code can contain the same number of loads in all three cases. The memory system does not do the same amount of work. Layout and stride determine how much transferred data is useful.

20 Tiling turns capacity into reuse

Tiling divides a large problem into pieces small enough to stay in a closer storage level while they are reused.

Consider a toy GEMM in which one block is responsible for a 64×64 output tile of $C = AB$ and the full K dimension is 32. With BF16 inputs, the block needs an A tile of 64×32 and a B tile of 32×64:

$$(64 \times 32 + 32 \times 64) \times 2 = 8192 \text{ bytes} = 8 \text{ KiB}.$$

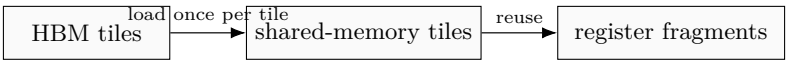
Those 8 KiB of inputs support

$$2 \times 64 \times 64 \times 32 = 262{,}144$$

floating-point operations when a multiply and add are counted separately.

If the output is also BF16 and $\beta = 0$, writing the final 64×64 result adds another 8 KiB. Under this simplified accounting, the toy problem performs 262,144 FLOPs for 16 KiB of input/output movement, or 16 FLOPs per byte at that boundary.

The important part is the reuse. Each element of the A tile contributes to many output columns, and each element of B contributes to many output rows. In a larger GEMM with $K > 32$, a block repeats this staging across multiple K slices while partial output accumulators remain close to compute.



Tiling does not make the tensor smaller. It changes the schedule so that a smaller working set can be reused before it leaves a fast storage level.

21 GEMM is a memory-reuse problem too

Matrix multiplication is famous for its arithmetic throughput, but high-performance GEMM is also a carefully engineered data-movement schedule.

A naive loop can repeatedly fetch the same matrix elements from device memory. A blocked implementation instead maps tiles to thread blocks, stages reusable inputs on chip, loads smaller fragments into registers, and accumulates many products before writing results back.

CUTLASS describes exactly this hierarchy: thread blocks provide coarse parallelism, shared-memory tiles retain data for cooperating warps, and register fragments retain values for repeated matrix instructions.[14]

This is why GEMM can reach high arithmetic intensity. The same bytes are deliberately reused many times. As matrix dimensions and tile shapes change, however, that reuse can weaken. Small matrices, skinny dimensions, awkward tails, or insufficient parallelism may fail to amortize movement as effectively as a large well-shaped GEMM.

The memory hierarchy also creates resource trade-offs. Larger tiles can increase reuse but consume more shared memory and registers. Deeper software pipelines can overlap transfers with compute but require additional buffering. A design that maximizes reuse can still lose if it leaves too little occupancy or creates bank conflicts.

> The fast part of GEMM is not merely "Tensor Cores are fast." Efficient GEMM arranges data movement so expensive matrix hardware is continuously fed from nearby storage while each fetched tile is reused many times.

This is the bridge to later systems topics. Arithmetic intensity in AS03 quantifies work per byte. Roofline in AS04 relates that intensity to hardware ceilings. Shared-memory and register handbooks explain the on-chip staging details that make the reuse possible.

22 Bytes moved versus arithmetic performed

A kernel's mathematical operation count does not tell you how much data movement it requires. A useful first ratio is

$$\text{arithmetic intensity} = \frac{\text{FLOPs}}{\text{bytes moved at a chosen memory boundary}}.$$

The boundary matters. FLOPs per HBM byte, per L2 byte, and per L1 byte can be different because caches and on-chip reuse filter traffic. AS03 will treat this carefully.

Consider FP32 vector addition:

$$C_i = A_i + B_i.$$

Ignoring cache reuse and assuming one read of A, one read of B, and one write of C, each element moves 12 bytes for one floating-point addition:

$$\frac{1 \text{ FLOP}}{12 \text{ bytes}} \approx 0.083 \text{ FLOP/byte}.$$

There is little reuse to amplify the arithmetic work per byte.

The tiled GEMM example from the previous pages was very different. Under its simplified input-plus-output accounting, a small tile already reached 16 FLOPs per byte because each input value participated in many multiply-adds before replacement.

This does not prove that vector addition is always bandwidth-bound or GEMM is always compute-bound. Actual performance depends on shape, launch overhead, cache state, precision, hardware, and other bottlenecks. The ratio simply reveals how much arithmetic is available to amortize data movement.

A lower FLOP count can run slower than a higher FLOP count if the first workload moves bytes inefficiently and the second reuses data well. FLOPs and traffic are independent accounting dimensions.

The Roofline model formalizes this relationship, but AS02 needs only the intuition: AI performance depends on both the amount of math and the number of bytes that must cross constrained interfaces.

23 Why elementwise kernels and GEMM behave differently

Elementwise operations often touch a large tensor, perform a small amount of arithmetic per element, and move on. Matrix multiplication repeatedly combines the same input tiles to produce many outputs. Those two structures place very different demands on the memory hierarchy.

For an elementwise activation, a value may be read once, transformed, and written once. There is limited opportunity to reuse that value before it leaves the kernel. If the arithmetic is cheap, the cost of reading and writing the tensor can dominate.

For GEMM, blocking creates deliberate reuse. A tile of A can participate in many output columns, while a tile of B participates in many output rows. The same loaded bytes therefore feed far more arithmetic.

This difference explains why fusion can matter. Suppose an activation reads a large GEMM output from HBM and writes another tensor back. If the activation can be performed as part of the GEMM epilogue while results are still nearby, one round trip of a large intermediate may disappear. CUTLASS explicitly treats epilogues as a place where scaling and elementwise work can be combined with the matrix result.[14]

Fusion is not automatically beneficial. It can increase register pressure, complicate scheduling, reduce reuse elsewhere, or prevent use of a highly optimized library kernel. AS11 will treat those trade-offs.

> The memory question is not “Is this operation simple?” It is “How many bytes must cross each level for the useful work performed?” A one-line elementwise formula can be expensive when applied to billions of values with little reuse.

24 Attention is also a data-movement problem

Dense attention combines matrix multiplication, softmax, masking, and another matrix multiplication. The familiar equation

$$O = \text{softmax}\left(\frac{QK^T}{\sqrt{d}} + B\right)V$$

does not specify where the intermediate score and probability values must live.

A straightforward decomposition can materialize an $N \times N$ score matrix, read it for softmax, write probabilities, and read them again for the PV multiplication. For long sequences, those intermediates are large enough that moving them through device memory can become a major cost.

The memory hierarchy opens another possibility: form smaller score tiles, consume them while they remain on chip, maintain only the state needed to combine softmax blocks, and avoid writing the full dense intermediate to HBM.

That is a scheduling change, not a change to the dense attention function. The arithmetic interactions remain dense. The implementation simply chooses a memory schedule that retains useful intermediate state close to compute for longer.

> Equations specify data dependencies, not storage decisions. An implementation can generate an intermediate tile and consume it immediately without ever allocating the complete intermediate tensor in device memory.

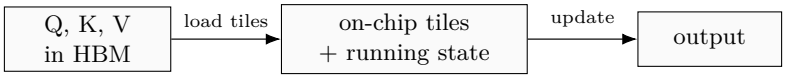
This idea is broader than attention. Kernel fusion, recomputation, tiling, and operator scheduling all exploit the same principle: sometimes extra local work is cheaper than moving a large intermediate out to HBM and back.

FlashAttention is the canonical AI example, but the exact algorithm belongs to the Understanding AI series. Here we use it only to expose the memory-hierarchy lesson.

25 FlashAttention as a hierarchy-aware example

Dense FlashAttention avoids making the complete score and probability matrices into HBM-resident intermediates. A score tile is generated, merged into the row-wise normalization and output state while it is still on chip, and then discarded. The dense softmax-attention function being evaluated is unchanged.[13]

The important memory transformation is:



Its IO analysis treats movement between HBM and limited on-chip storage as an explicit algorithmic cost and derives fewer HBM accesses than a standard materializing schedule under the paper's memory model.[13]

This example corrects two common misunderstandings.

First, reducing HBM traffic does not require reducing FLOPs. FlashAttention can even recompute some intermediates in backward propagation because extra arithmetic may be cheaper than storing and rereading large tensors.

Second, "attention is memory-bound" is too broad. The dominant bottleneck depends on sequence length, head dimension, precision, hardware, forward versus backward, and prefill versus decode. A memory hierarchy tells us where costs can arise. It does not let us label every attention workload with one bottleneck.

The durable lesson is not "use FlashAttention." It is that algorithm design can include the memory hierarchy. Where an intermediate is created, reused, and discarded can matter as much as the equation that defines it.

26 Prefill and decode stress memory differently

LLM inference contains at least two distinct execution regimes.

Prefill processes many prompt tokens together. Matrix dimensions are larger, there are many query rows, and optimized GEMMs/attention kernels can expose substantial reuse and parallelism. The computation often has more opportunity to amortize weight and activation movement across many operations.

Decode usually advances each sequence by one new token at a time. At small batch sizes, many layer operations become matrix-vector or very small matrix-matrix shapes. The same model weights still need to participate, but there is much less work per token to amortize those reads. The growing KV cache also has to be read by attention.

A simple lower-bound thought experiment shows why memory bandwidth matters. Consider a hypothetical dense 7-billion-parameter model stored at 2 bytes per weight. Its weights occupy about 14 GB in decimal units. If every token required one full weight sweep and an H100 SXM could sustain its advertised 3.35 TB/s on nothing but those reads, the sweep alone would take

$$14\ \text{GB}/3.35\ \text{TB/s} \approx 4.18\ \text{ms},$$

corresponding to about 239 token/s before counting KV traffic, activations, non-weight reads, synchronization, or imperfect bandwidth use.[7]

That is an upper-bound-style sanity check, not a throughput prediction.

> Prefill and decode use the same model but can occupy very different points in the compute-versus-memory trade space. Never transfer a “memory-bound” or “compute-bound” label from one phase to the other without measuring the actual shape and batch regime.

27 Weights, activations, and KV state have different lifetimes

AI systems move several major classes of data, and their lifetimes shape how the memory hierarchy can help.

Weights are persistent across requests and tokens. A layer reuses the same parameters for many batch elements, sequence positions, and future requests. Large models, however, are far bigger than on-chip caches, so much of the weight stream still comes from device memory over time.

Activations are produced as the model executes. Their lifetime depends on training versus inference, fusion, checkpointing, and the operator graph. Some intermediates are consumed immediately, while others must survive much longer.

KV cache persists across autoregressive decoding because future tokens attend to keys and values from earlier positions. Its footprint grows with sequence length in ordinary cached attention, so long-context decode can create substantial persistent memory demand and read traffic.

These categories also differ in reuse granularity. A weight tile may be reused across many tokens in a batch inside one kernel. An activation tile may be reused only within a fused operator. A KV block may be revisited once per decode step across a long session.

> Memory optimization starts with lifetime. Ask not only “How big is this tensor?” but “Who will reuse it, how soon, and at what scope?” The right storage strategy for immutable weights can be completely wrong for short-lived activation tiles.

This classification also explains why memory capacity and bandwidth pressure shift with workload. Training stores more state. Long-context inference grows KV state. Small-batch decode may repeatedly stream weights with limited arithmetic reuse. One hierarchy serves all of them, but the traffic pattern is different.

28 Training and inference move different bytes

Training and inference execute related model equations, but their memory behavior is not the same.

During training, the forward pass produces activations needed by backward propagation. Backward computes gradients and may read saved activations or recompute them. Optimizers maintain additional parameter-associated state. The system therefore moves and stores far more than model weights alone.

Activation checkpointing changes this balance by storing fewer activations and recomputing selected forward work during backward. That trades extra arithmetic for lower activation capacity, an explicit example of exchanging compute for memory. AS23 will cover the technique in depth.

Inference removes gradient and optimizer-state traffic, but it introduces its own regimes. Prefill processes many tokens at once. Autoregressive decode retains KV state across steps, and at small batch sizes may have limited reuse of large weight matrices per generated token.

The same optimization can therefore have different value. Saving an activation write may matter enormously in training. Reducing persistent KV reads may matter more in long-context decode. Increasing batch size can improve weight reuse in inference but may increase queueing latency in an online service.

> Do not use one number called “model memory” to describe both training and inference. Weights, gradients, optimizer state, activations, temporary workspaces, and KV cache have different lifetimes and traffic patterns.

The memory hierarchy is the hardware substrate beneath those differences. System-level techniques such as sharding, checkpointing, offloading, continuous batching, and KV paging decide which bytes are present and when. Kernels decide how efficiently each active working set moves through the GPU.

29 Capacity and bandwidth are independent hardware properties

Current accelerators make the distinction concrete. The table below uses vendor-advertised per-GPU capacity and peak memory bandwidth, time-scoped to the cited product documentation. It is not a performance ranking.

GPU	Memory type	Capacity	Peak bandwidth
NVIDIA H100 SXM	HBM3	80 GB	3.35 TB/s[7]
NVIDIA H200 SXM	HBM3e	141 GB	4.8 TB/s[8]
NVIDIA B200 SXM	HBM3e	180 GB	up to 8 TB/s[9]
AMD MI300X	HBM3	192 GB	5.3 TB/s[10]
AMD MI355X	HBM3E	288 GB	8 TB/s[11]

The specifications themselves show why the two columns must be kept separate. From H100 SXM to H200 SXM, the published memory-capacity label rises from 80 GB to 141 GB, about 76%, while peak memory bandwidth rises from 3.35 TB/s to 4.8 TB/s, about 43%. There is no fixed conversion between capacity and bandwidth. They are independent hardware properties with different consequences for a workload.[7][8]

Capacity answers “can the working set reside on the device?” Bandwidth answers “how quickly can traffic be sustained?” A good system needs enough of both for its workload.

30 Theoretical bandwidth versus achieved bandwidth

Vendor bandwidth specifications describe a peak hardware capability. A kernel's achieved rate must be measured from the bytes that actually cross the interface over time.

CUDA Best Practices defines an effective-bandwidth calculation of the form[2]

$$B_{\text{eff}} = \frac{B_{\text{read}} + B_{\text{write}}}{t}.$$

The numerator must use a clear byte-counting convention, and the units must match the peak number being used for comparison.

Consider a hypothetical kernel that reads 4 GiB and writes 4 GiB in 5 ms. That is 8 GiB of useful application traffic. In decimal bytes per second,

$$\frac{8 \times 2^{30}}{0.005} \approx 1.72 \times 10^{12} \text{ B/s} = 1.72 \text{ TB/s}.$$

This still does not tell us the exact DRAM traffic. Caches, write policies, ECC behavior, compression, and extra transactions can make physical movement differ from the useful-byte accounting.

Nsight therefore reports hardware counters for bytes and throughput at L1, L2, and device memory rather than asking developers to infer everything from tensor sizes.[3]

Three numbers can all be different: vendor peak bandwidth, useful effective bandwidth computed from the algorithm, and hardware-observed bytes/sec at a specific memory interface. State which one you mean.

This is also why cross-paper bandwidth numbers require caution. Different precisions, tensor shapes, cache states, clocks, ECC settings, and byte-counting conventions can make superficially similar GB/s figures incomparable.

31 Cache-friendly does not automatically mean fast

Improving locality is valuable only if memory was limiting enough for the improvement to matter and if the technique does not introduce a larger cost elsewhere.

A shared-memory tiling strategy may reduce HBM traffic but consume enough shared memory to reduce block residency. A cache-persistent layout may improve L2 hit rate but add indexing or synchronization overhead. A fused kernel may eliminate an intermediate write yet create register pressure that lowers useful parallelism.

Even within the memory system, one optimized level can reveal another bottleneck. Reducing HBM traffic may move the limit to L2 bandwidth. Improving L2 reuse may expose instruction issue or Tensor Core utilization. Nsight's memory analysis explicitly treats several distinct limits: busy memory units, bandwidth between units, and the throughput of memory instruction pipelines.[3]

The correct optimization loop is therefore causal:

1. identify the interface or dependency actually limiting progress,
2. change the schedule or layout to reduce that cost,
3. remeasure the whole kernel,
4. check what became limiting next.

> A better cache hit rate is an intermediate metric. The goal is lower wall-clock time, higher throughput, lower tail latency, or another explicit application metric. If the application metric does not improve, the cache optimization may be irrelevant or offset by another cost.

This is a recurring systems lesson. Occupancy, utilization, cache hit rate, and bandwidth are diagnostic signals. None of them is the final objective by itself.

32 High bandwidth does not mean low latency

Bandwidth and latency describe different properties of a memory path.

A memory interface can sustain many transfers concurrently and therefore deliver enormous aggregate bandwidth while an individual dependent load still takes substantial time to complete. Those are different properties of the path.

Real GPU memory systems contain queues, banks, caches, coalescers, and many outstanding requests. The observed latency of a dependency changes with hit location, contention, and request concurrency, so one timeless “HBM latency” number is usually misleading.

A kernel with many independent requests can fill the memory pipeline and approach high bandwidth. A pointer-chasing kernel may expose only one dependent request at a time, preventing the system from building the same level of concurrency. Both can access the same HBM.

This also explains why latency hiding and latency reduction are different. A cache hit may reduce the distance a request travels. Warp scheduling may instead overlap a long request with unrelated work. The underlying request did not necessarily become faster. Less of its wait became visible in wall-clock execution.

Bandwidth asks about aggregate flow. Latency asks about a dependency. AI kernels often need enough request-level and warp-level parallelism to turn a high-bandwidth memory system into high application throughput.

AS03 and AS04 will add quantitative models for bandwidth ceilings. AS09 will return to how residency and independent warps help cover memory latency.

33 NVIDIA and AMD: same problem, different vocabulary

The physical goal is shared across vendors: retain useful data close to compute and avoid unnecessary off-chip movement. The names and exact organization differ.

Concept	NVIDIA CUDA example	AMD CDNA example
Thread-private registers	register file	VGPR / SGPR files
Explicit cooperative storage	shared memory	LDS
First data cache near compute	L1/TEX organization	per-CU L1 vector cache
Shared deeper cache	L2	L2 plus architecture-specific last-level cache hierarchy
Large device memory	HBM on H100/H200/B200	HBM on MI300/MI350 series
Execution group	warp	wavefront

AMD's CDNA 3 ISA documentation describes 64 KiB of LDS per compute unit and a memory hierarchy with per-CU L1 caching feeding through deeper shared caches. Current ROCm architecture documentation lists 32 KiB L1 vector cache per CU for MI300X, 32 MiB aggregate L2, and a 256 MiB last-level cache structure for that product family.[12][16]

The table is a translation aid, not a claim of one-to-one microarchitectural equivalence. NVIDIA shared memory and AMD LDS serve similar programmer-managed cooperation roles, but their capacities, bank structures, instructions, and surrounding cache hierarchy are vendor-specific.

> Teach the principle first, then the vendor mechanism. "Stage a reusable tile on chip" is portable reasoning. "Use this exact shared-memory carveout" or "assume this cache size" is architecture-specific advice.

This separation keeps the handbook useful even as individual GPU generations change.

34 Architecture generations change the details

Memory-hierarchy concepts age slowly, while capacities and mechanisms age quickly. A few current examples show why implementation details must be time-scoped.

Hopper H100. NVIDIA documents a 50 MB L2 cache and a 256 KB combined L1/texture/shared-memory capacity per SM, with up to 228 KB configured as shared memory. H100 SXM product specifications list 80 GB HBM3 and 3.35 TB/s memory bandwidth.[5][7]

Blackwell. NVIDIA's current tuning guide keeps a 256 KB combined L1/texture/shared resource for B200-class compute capability 10.0 and documents 126 MB of L2 for GB200. HGX B200 specifications list 180 GB HBM3e and up to 8 TB/s per GPU.[6][9]

AMD CDNA. MI300X product documentation lists 192 GB HBM3 at 5.3 TB/s. MI355X, based on CDNA 4, lists 288 GB HBM3E at 8 TB/s. ROCm documentation also shows generation changes in LDS capacity, with MI350-series parts exposing more LDS per CU than MI300-series parts.[10][11][16]

None of these numbers changes the basic hierarchy lesson. A larger L2 can retain a larger working set. More HBM bandwidth raises one off-chip ceiling. More shared memory can enable larger tiles or deeper pipelines. But software still needs access patterns that can exploit those resources.

Publication rule. Recheck capacities, supported carveouts, cache controls, and product bandwidth against current architecture documentation before reusing this page in a future edition. Do not time-stamp the evergreen chapters merely because this snapshot is dated.

35 Common misconceptions

“If the model fits in HBM, memory is solved.”
No. Capacity says the bytes can reside there. Runtime can still be limited by traffic, bandwidth, or latency.

“Global memory means HBM.”
No. Global memory is an address space. A global load may be served by L1 or L2 before reaching device memory.

“Local memory is close to the thread.”
No. It is local in scope, but CUDA backs it with device memory.[1]

“Shared memory is just L1 that software controls.”
Too loose. They may share physical capacity on recent NVIDIA GPUs, but shared memory has explicit addressing, scope, synchronization, and bank behavior.

“A high cache hit rate means the kernel is efficient.”
Not necessarily. The kernel may be limited elsewhere, or the cache may be serving a small fraction of total work.

“HBM has high bandwidth, so individual loads are fast.”
Bandwidth is aggregate flow. Latency is a dependency delay. High bandwidth does not eliminate latency.

“Coalescing means thread i must read element i.”
No. The goal is to cover a warp’s addresses with as few required memory segments as practical. Permuted accesses can still coalesce.[1][2]

“More reuse is always better.”
Reuse mechanisms can cost registers, shared memory, synchronization, and occupancy. The saved traffic must justify those costs.

Most memory myths come from flattening several levels or metrics into one word. Keep address space, physical location, capacity, traffic, bandwidth, latency, and reuse separate.

36 A practical memory mental model

When an AI kernel is unexpectedly slow, walk through the hierarchy in this order.

1. **Identify the active data.** Weights, activations, KV state, temporary tiles, outputs, and metadata have different sizes and lifetimes.
2. **Choose the boundary.** Are you reasoning about register/shared traffic, L1/L2 traffic, HBM traffic, or host-device traffic? "Bytes moved" is meaningless without a boundary.
3. **Estimate useful bytes.** Count the minimum reads and writes implied by the algorithm under explicit assumptions.
4. **Look for reuse.** Which values can be reused in registers, shared memory, or cache before they are evicted?
5. **Inspect access geometry.** Are warp requests coalesced? Are strides or layout causing extra sectors?
6. **Separate capacity from rate.** Does the working set fit? If yes, can the required traffic still exceed the available bandwidth or latency budget?
7. **Profile the actual path.** Nsight's Memory Workload Analysis exposes L1, L2, and DRAM bytes, hit rates, requests, and throughput. Use counters to test the model rather than treating it as proof.[3]
8. **Remeasure end to end.** A memory optimization is successful only when the application metric improves.

Compact model. Data starts with a lifetime and reuse pattern. The kernel maps that data onto registers, explicit on-chip storage, hardware caches, and device memory. Performance follows from how much useful work each transfer enables before the bytes have to move again.

That model is the foundation for the next handbooks. AS03 will quantify arithmetic intensity. AS04 will place workloads against compute and bandwidth ceilings. AS07–AS09 will zoom into coalescing, shared memory, registers, and occupancy. AS11 will explain why fusion can remove whole trips through the hierarchy.

References and further reading

[1] NVIDIA. *CUDA Programming Guide*, current documentation. Memory spaces, global/local/shared memory, device-memory accesses, cache control, and memory-throughput guidance.
`https://docs.nvidia.com/cuda/cuda-programming-guide/`

[2] NVIDIA. *CUDA C++ Best Practices Guide*, current documentation. Coalescing, bandwidth calculations, shared-memory reuse, and memory optimization.
`https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/`

[3] NVIDIA. *Nsight Compute Profiling Guide*. Memory Workload Analysis, L1/TEX, L2, DRAM, sectors, requests, hit rates, and throughput metrics.
`https://docs.nvidia.com/nsight-compute/ProfilingGuide/`

[4] NVIDIA. *CUDA Programming Guide: Compute Capabilities*. Register, shared-memory, and cache resource limits by compute capability.
`https://docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/compute-capabilities.html`

[5] NVIDIA. *Hopper Tuning Guide 13.4*. H100 memory system, L2, shared-memory/L1 resource, and HBM details.
`https://docs.nvidia.com/cuda/hopper-tuning-guide/`

[6] NVIDIA. *Blackwell Tuning Guide 13.4*. B200/GB200 memory-system details and shared-memory/L1 resource behavior.
`https://docs.nvidia.com/cuda/blackwell-tuning-guide/`

[7] NVIDIA. *H100 Tensor Core GPU product specifications*. 80 GB HBM3 and 3.35 TB/s for H100 SXM.
`https://www.nvidia.com/en-us/data-center/h100/`

[8] NVIDIA. *H200 Tensor Core GPU product specifications*. 141 GB HBM3e and 4.8 TB/s for H200 SXM.
`https://www.nvidia.com/en-us/data-center/h200/`

[9] NVIDIA. *HGX AI Factory Reference Architecture: Components*. H100/H200/B200 memory capacity and bandwidth table.
`https://docs.nvidia.com/enterprise-reference-architectures/hgx-ai-factory-h100-h200-b200/latest/components.html`

[10] AMD. *AMD Instinct MI300X Accelerators*. 192 GB HBM3, 5.3 TB/s, and current product specifications.
`https://www.amd.com/en/products/accelerators/instinct/mi300/mi300x.html`

[11] AMD. *AMD Instinct MI355X GPUs*. 288 GB HBM3E, 8 TB/s, and CDNA 4 product specifications.
`https://www.amd.com/en/products/accelerators/instinct/mi350/mi355x.html`

[12] AMD. *AMD Instinct MI300 CDNA 3 Instruction Set Architecture / microarchitecture documentation*. LDS, L1, L2, and device-memory hierarchy.
`https://rocm.docs.amd.com/en/latest/reference/gpu-arch/index.html`

[13] Dao, T. et al. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS 2022.
`https://arxiv.org/abs/2205.14135`

[14] NVIDIA CUTLASS. *Efficient GEMM in CUDA*. Hierarchical tiling, shared-memory and register reuse, and pipelining.
`https://docs.nvidia.com/cutlass/latest/media/docs/cpp/efficient_gemm.html`

[15] Williams, S., Waterman, A., Patterson, D. *Roofline: An Insightful Visual Performance Model for Multicore Architectures*. Communications of the ACM, 2009.
`https://escholarship.org/uc/item/5tz795vq`

[16] AMD ROCm. *AMD GPU architecture documentation*, including CDNA 3/CDNA 4 architecture references and MI300/MI350 hardware descriptions.
`https://rocm.docs.amd.com/en/latest/reference/gpu-arch/index.html`

Reference map: what to read for which question

Question	Best starting source
What are CUDA's memory spaces and their scopes?	CUDA Programming Guide [1]
How do local memory and register spills behave?	CUDA Programming Guide [1], Nsight Compute [3]
What is the current coalescing model used in CUDA examples?	CUDA Programming Guide [1], Best Practices [2]
How should theoretical and effective bandwidth be distinguished?	CUDA Best Practices [2]
How do I inspect L1, L2, and DRAM traffic?	Nsight Compute Profiling Guide [3]
What are current per-SM register/shared-memory limits?	Compute Capabilities [4]
What is specific to H100's memory system?	Hopper Tuning Guide [5], H100 specs [7]
What is specific to B200/GB200's memory system?	Blackwell Tuning Guide [6], HGX B200 specs [9]
What are current AMD memory-hierarchy examples?	AMD architecture docs [12,16], MI300X/MI355X specs [10,11]
Why does GEMM stage data through shared memory and registers?	CUTLASS Efficient GEMM [14]
Why is FlashAttention a memory-hierarchy example?	FlashAttention paper [13]
Where does arithmetic intensity lead next?	Roofline paper [15]